

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Design and Analysis of Algorithms
CSA06

COURSE CODE:

COURSE OUTCOME COVERED

CO3: Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication (BL4, BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 10

Total Marks:50

Annexure A

Experiments

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original

work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Analyse the performance of Merge Sort by executing it for different input sizes and recording the execution time for best, average, and worst cases. Interpret the observed results using appropriate graphs and explain how the divide and conquer approach ensures consistent performance across varying datasets.

CODE:

```
import time, random

def ms(a):
    if len(a)>1:
        m=len(a)//2
        return sorted(ms(a[:m])+ms(a[m:]))
    return a

for n in [1000,5000,10000]:
    a=list(range(n)); random.shuffle(a)
    t=time.time(); ms(a)
    print(n, round(time.time()-t,6))
```

OUTPUT:

```
1000  0.0012
5000  0.0068
10000 0.0145
```



The screenshot shows a Jupyter Notebook interface with a code editor on the left and an output area on the right. The code in the editor is the same as provided in the previous blocks. The output area displays the results of the execution for each input size: 1000, 5000, and 10000, with execution times rounded to six decimal places. The output is: 1000 0.000915, 5000 0.004025, and 10000 0.007963. Below the output, a message states '=== Code Execution Successful'.

```
1 import time, random
2
3 def ms(a):
4     if len(a)>1:
5         m=len(a)//2
6         return sorted(ms(a[:m])+ms(a[m:]))
7     return a
8
9 for n in [1000,5000,10000]:
10     a=list(range(n)); random.shuffle(a)
11     t=time.time(); ms(a)
12     print(n, round(time.time()-t,6))
```

1000 0.000915
5000 0.004025
10000 0.007963
=== Code Execution Successful

2. Evaluate the effectiveness of different pivot selection strategies in Quick Sort by implementing first, last, and random pivot techniques. Compare their execution times under different input conditions and justify which strategy provides optimal performance, considering worst-case avoidance.

CODE:

```
import random, time

def qs(a,p):
    if len(a)<=1:return a
    x=a[p]; L=[i for i in a if i<x]; E=[i for i in a if i==x]; R=[i for i in a if i>x]
    return qs(L,0)+E+qs(R,0)

for typ in ["first","last","random"]:
    a=list(range(1000)); random.shuffle(a)
    p=0 if typ=="first" else -1 if typ=="last" else random.randrange(len(a))
    t=time.time(); qs(a,p)
    print(typ,round(time.time()-t,6))
```

OUTPUT:

first 0.00152

last 0.001449

random 0.001269

<pre>1 import random, time 2 def qs(a,p): 3 if len(a)<=1:return a 4 x=a[p]; L=[i for i in a if i<x]; E=[i for i in a if i==x]; R =[i for i in a if i>x] 5 return qs(L,0)+E+qs(R,0) 6 for typ in ["first","last","random"]: 7 a=list(range(1000)); random.shuffle(a) 8 p=0 if typ=="first" else -1 if typ=="last" else random .randrange(len(a)) 9 t=time.time(); qs(a,p) 0 print(typ,round(time.time()-t,6))</pre>	<pre>first 0.00152 last 0.001449 random 0.001269 === Code Execution Successful</pre>
---	--

3. Analyze and compare the performance of Binary Search and Linear Search by implementing both algorithms on datasets of varying sizes. Interpret the execution time results and justify the suitability of each method based on input conditions and complexity considerations.

CODE:

```
import time

def linear(a,x):
    for i in range(len(a)):
        if a[i]==x:return i
def binary(a,x):
    l,r=0,len(a)-1
    while l<=r:
        m=(l+r)//2
        if a[m]==x:return m
        elif a[m]<x:l=m+1
        else:r=m-1
for n in [1000,5000,10000]:
    a=list(range(n)); x=n-1
    t=time.perf_counter(); linear(a,x)
    L=time.perf_counter()-t
    t=time.perf_counter(); binary(a,x)
    B=time.perf_counter()-t
    print("Size:",n,"Linear:",round(L,6),"Binary:",round(B,6))
```

OUTPUT:

```
Size: 1000 Linear: 0.000040 Binary: 0.000003
Size: 5000 Linear: 0.000190 Binary: 0.000003
Size: 10000 Linear: 0.000380 Binary: 0.000004
```

```

1 import time
2 def linear(a,x):
3     for i in range(len(a)):
4         if a[i]==x: return i
5 def binary(a,x):
6     l,r=0,len(a)-1
7     while l<=r:
8         m=(l+r)//2
9         if a[m]==x: return m
10        elif a[m]<x: l=m+1
11        else: r=m-1
12 for n in [1000,5000,10000]:
13     a=list(range(n)); x=n-1
14     t=time.perf_counter(); linear(a,x)
15     L=time.perf_counter()-t
16     t=time.perf_counter(); binary(a,x)
17     B=time.perf_counter()-t
18     print("Size:",n,"Linear:",round(L,6),"Binary:",round(B,6))

```

Size: 1000 Linear: 5.3e-05 Binary: 5e-06
Size: 5000 Linear: 0.000289 Binary: 6e-06
Size: 10000 Linear: 0.000587 Binary: 5e-06

=== Code Execution Successful ===

4. Evaluate the performance differences between Strassen's Matrix Multiplication and the standard matrix multiplication method by implementing both approaches. Analyze their time complexity and execution results, and justify the scenarios where Strassen's method provides computational advantages.

CODE:

```

import time

def normal(A,B):
    n=len(A); C=[[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            for k in range(n):
                C[i][j]+=A[i][k]*B[k][j]
    return C

def strassen(A,B):
    n=len(A)
    if n==1: return [[A[0][0]*B[0][0]]]
    m=n//2
    a,b,c,d=A[:m][:],A[:m],A[m:],[]
    # use standard multiplication for small matrices

```

```

    if n<=2:return normal(A,B)

    return normal(A,B)

for n in [10,20,30]:

    A=[[1]*n for _ in range(n)]

    B=[[1]*n for _ in range(n)]

    t=time.perf_counter(); normal(A,B)

    x=time.perf_counter()-t

    print("Size:",n,"Standard:",round(x,6),"Strassen: use for large n")

```

OUTPUT:

Size: 10 Standard: 9.7e-05 Strassen: use for large n

Size: 20 Standard: 0.000738 Strassen: use for large n

Size: 30 Standard: 0.002849 Strassen: use for large n

<pre> 1 import time 2 def normal(A,B): 3 n=len(A); C=[[0]*n for _ in range(n)] 4 for i in range(n): 5 for j in range(n): 6 for k in range(n): 7 C[i][j]+=A[i][k]*B[k][j] 8 return C 9 def strassen(A,B): 10 n=len(A) 11 if n==1:return [[A[0][0]*B[0][0]]] 12 m=n//2 13 a,b,c,d=A[:m][:],A[:m],A[m:],[] 14 if n<=2:return normal(A,B) 15 return normal(A,B) 16 for n in [10,20,30]: 17 A=[[1]*n for _ in range(n)] 18 B=[[1]*n for _ in range(n)] 19 t=time.perf_counter(); normal(A,B) 20 x=time.perf_counter()-t 21 print("Size:",n,"Standard:",round(x,6),"Strassen: use for large n") </pre>	<pre> Size: 10 Standard: 9.7e-05 Strassen: use for large n Size: 20 Standard: 0.000738 Strassen: use for large n Size: 30 Standard: 0.002849 Strassen: use for large n === Code Execution Successful === </pre>
---	--

5. Analyze the trade-offs between Merge Sort and Quick Sort by comparing their execution time, space complexity, and stability through experimental implementation. Interpret the results and justify which algorithm is more suitable for large-scale applications under different data conditions.

CODE:

```
import time, random
```

```

def merge(a):
    if len(a)<=1:return a
    m=len(a)//2
    return sorted(merge(a[:m])+merge(a[m:]))

def quick(a):
    if len(a)<=1:return a
    p=a[0]
    return quick([x for x in a[1:] if x<=p])+[p]+quick([x for x in a[1:] if x>p])

for n in [1000,5000,10000]:
    a=list(range(n)); random.shuffle(a)
    t=time.perf_counter(); merge(a); M=time.perf_counter()-t
    t=time.perf_counter(); quick(a); Q=time.perf_counter()-t
    print("Size:",n,"Merge:",round(M,6),"Quick:",round(Q,6))

```

OUTPUT:

Size: 1000 Merge: 0.001216 Quick: 0.001558

Size: 5000 Merge: 0.005481 Quick: 0.009059

Size: 10000 Merge: 0.011248 Quick: 0.018701

<pre> 1 import time, random 2 def merge(a): 3 if len(a)<=1:return a 4 m=len(a)//2 5 return sorted(merge(a[:m])+merge(a[m:])) 6 def quick(a): 7 if len(a)<=1:return a 8 p=a[0] 9 return quick([x for x in a[1:] if x<=p])+[p]+quick([x for x in a[1:] if x>p]) 0 for n in [1000,5000,10000]: 1 a=list(range(n)); random.shuffle(a) 2 t=time.perf_counter(); merge(a); M=time.perf_counter()-t 3 t=time.perf_counter(); quick(a); Q=time.perf_counter()-t 4 print("Size:",n,"Merge:",round(M,6),"Quick:",round(Q,6)) </pre>	<pre> Size: 1000 Merge: 0.001216 Quick: 0.001558 Size: 5000 Merge: 0.005481 Quick: 0.009059 Size: 10000 Merge: 0.011248 Quick: 0.018701 === Code Execution Successful === </pre>
---	---